

EVALUACIÓN	Obligatorio 2	GRUPO		FECHA	25/06/2026
MATERIA	Estructuras de Datos y Algoritmos 2				
CARRERA	Ingeniería en Sistemas				
CONDICIONES	- Puntaje máximo: 20 puntos - Puntaje mínimo: 0 puntos - Fecha de entrega: 25/06/2026 hasta las 21:00 horas en gestion.ort.edu.uy (max. 40Mb en formato zip) Uso de material de apoyo y/o consulta <u>Inteligencia Artificial Generativa</u> - Seguir las pautas de los docentes: Se deben seguir las instrucciones específicas de los docentes sobre cómo utilizar la IA en cada curso. - Citar correctamente las fuentes y usos de IA: Siempre que se utilice una herramienta de IA para generar contenido, se debe citar adecuadamente la fuente y la forma en que se utilizó. - Se debe indicar: a. la /las herramientas utilizadas y b. el contexto de uso: por ejemplo, generación de ideas, redacción inicial, análisis de datos, corrección de estilo, etc. - Todo contenido producido por IAG debe ser revisado y verificado; cualquier error presente será responsabilidad del estudiante. - Ser responsables con el uso de la IA: Conocer los riesgos y desafíos, como la creación de “alucinaciones”, los peligros para la privacidad, las cuestiones de propiedad intelectual, los sesgos inherentes y la producción de contenido falso. - El uso de herramientas de Inteligencia Artificial Generativa (IAG) puede emplearse como apoyo en el proceso de aprendizaje. Sin embargo, no sustituye el razonamiento crítico ni la elaboración personal. El estudiante es responsable de que el trabajo presentado refleje su propia comprensión, análisis y proceso cognitivo sobre el tema. - La defensa es obligatoria y eliminatoria. - La defensa será presencial salvo en casos en que se habilite la defensa oral sincrónica en forma remota. Defensa Fecha de defensa: 25/06/2026 <u>La defensa es obligatoria y eliminatoria.</u> El docente es quien definirá y comunicará la modalidad, y mecánica de defensa. <u>La no presentación a la misma implica la pérdida de la totalidad de los puntos del Obligatorio.</u> IMPORTANTE: 1. Inscribirse 2. Formar grupos de hasta 2 personas del mismo dictado 3. Subir el trabajo a Gestión antes de la hora indicada (ver hoja al final del documento: “RECORDATORIO”) Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con la Coordinadora de cursos o Coordinación Adjunta antes de las 20:00h del día de la entrega, a través del mail: adjuntos_ei@ort.edu.uy.				

Obligatorio 2

Ejercicio 6 - Monitoreo de focos críticos

Descripción

Una región contiene varios focos de radiación activos, cada uno ubicado en una posición del plano descrita por coordenadas cartesianas (x, y) . Un par de focos se considera **crítico** cuando la distancia euclídea entre ellos es menor o igual a un umbral D , ya que en ese caso pueden superponer sus dosis y requerir intervención inmediata.

Dado el conjunto de focos detectados, determine si existe al menos un par crítico.

La distancia entre dos focos (x_1, y_1) y (x_2, y_2) se calcula como:

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Entrada

- La primera línea contiene dos enteros N y D ($1 \leq N \leq 10^6$, $0 \leq D \leq 10^9$): la cantidad de focos detectados y la distancia umbral, respectivamente.
- Las siguientes N líneas contienen las coordenadas x e y de cada foco ($-10^9 \leq x, y \leq 10^9$).
- Los focos se listan en **orden arbitrario** y puede haber dos o más focos en la misma posición.

Salida

Imprima una única línea con `true` si existe al menos un par crítico, o `false` en caso contrario.

Restricciones

- Resolver utilizando una estrategia de **Divide and Conquer**.
- Complejidad esperada: $O(N \log N)$ en el peor caso.

Ejemplo

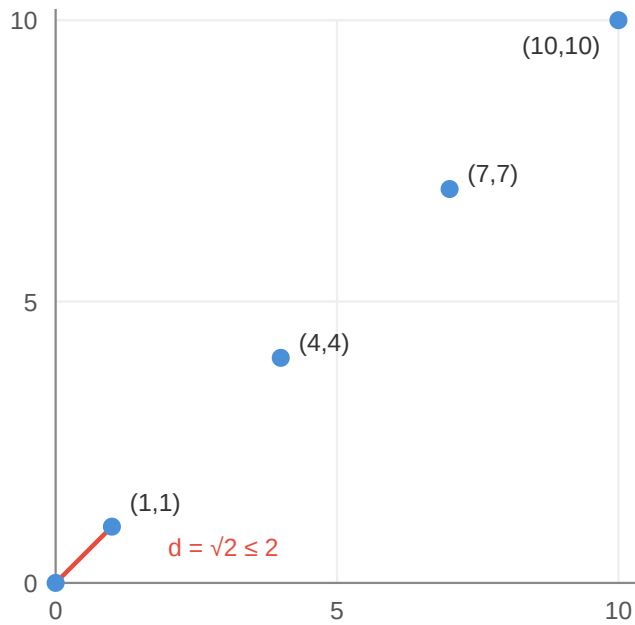
Input 1

```
5 2
0 0
1 1
4 4
7 7
10 10
```

Output 1

true

Explicación 1



Los focos en (0,0) y (1,1) están a distancia $\sqrt{2}$, que es menor que 2.

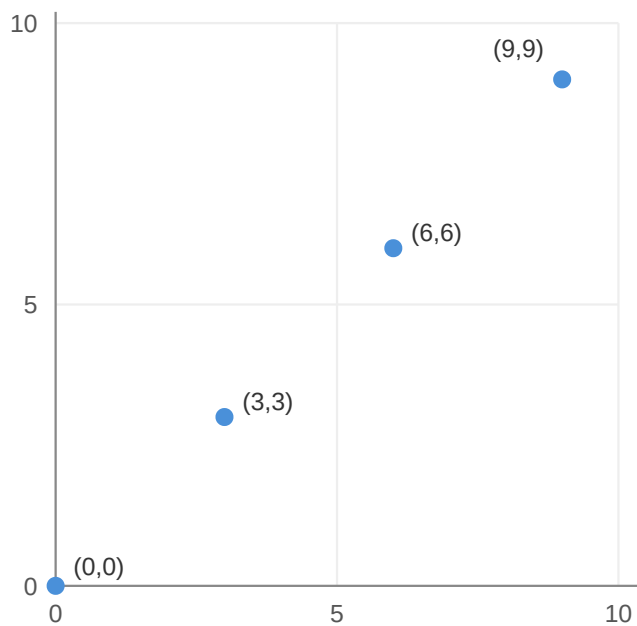
Input 2

```
4 1
0 0
3 3
6 6
9 9
```

Output 2

false

Explicación 2



Todos los focos están separados por distancias mayores que 1.

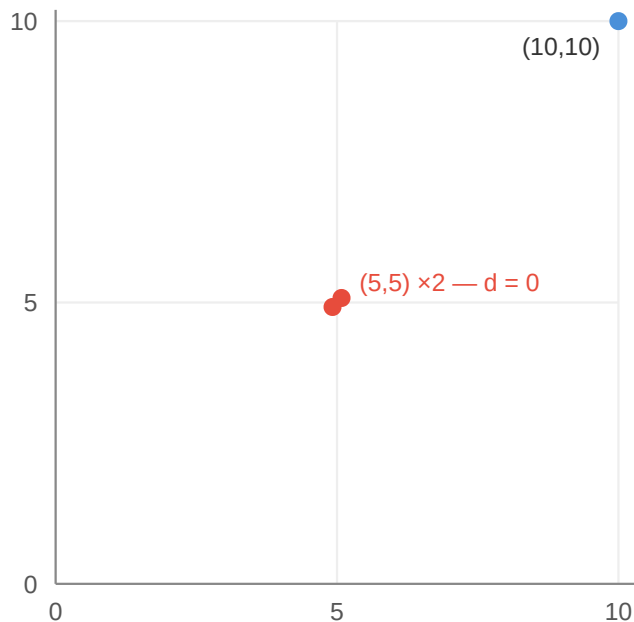
Input 3

```
3 0
5 5
5 5
10 10
```

Output 3

```
true
```

Explicación 3



Existen dos focos en exactamente la misma posición, por lo que la distancia entre ellos es 0.

Ejercicio 7 - Clasificación Iron Man

Descripción

El **Iron Man** es una triatlón que combina tres disciplinas: natación, ciclismo y carrera a pie. Por motivos de organización los competidores no comienzan al mismo tiempo, por lo que el primero en cruzar la meta no es necesariamente el ganador: el resultado se determina sumando los tiempos parciales de cada competidor en las tres disciplinas.

Dada la lista de tiempos de los L competidores, ordene a los competidores **de menor a mayor tiempo total** (menor tiempo equivale a mejor posición). El tiempo total del competidor i se calcula como

$$T_i = N_i + B_i + C_i.$$

Entrada

- La primera línea contiene un entero L ($1 \leq L \leq 10^6$), la cantidad de competidores.
- Las siguientes L líneas contienen los tiempos de **natación** ($N_1, \dots, N_L$), una por competidor.
- Las siguientes L líneas contienen los tiempos de **ciclismo** ($B_1, \dots, B_L$), una por competidor.
- Las siguientes L líneas contienen los tiempos de **carrera a pie** ($C_1, \dots, C_L$), una por competidor.

Cada tiempo es un entero $1 \leq T \leq 10^6$. Los competidores se identifican por su orden de entrada, comenzando en 1.

Salida

Imprima L líneas con los índices (1-indexados) de los competidores ordenados de menor a mayor tiempo total. Si dos competidores tienen el mismo tiempo total, deben aparecer en el mismo orden en que fueron ingresados.

Restricciones

- Resolver utilizando **merge sort** (ordenamiento por divide and conquer).
- Complejidad temporal: $O(L \log L)$ en el peor caso.
- Complejidad espacial: $O(L)$ memoria auxiliar.

Ejemplo

Input

```
3
10
12
9
20
18
25
30
28
35
```

Output

```
2
1
3
```

Explicación

Competidor	Natación	Ciclismo	Carrera	Total
1	10	20	30	60
2	12	18	28	58
3	9	25	35	69

Ordenando de menor a mayor tiempo total: **2** (58), **1** (60), **3** (69).

Ejercicio 8 - El cartel de neón

Descripción

Una tienda de letras luminosas recibe un pedido especial: un cliente quiere armar un cartel de neón usando exactamente las letras que tiene disponibles (representadas como una cadena de letras latinas minúsculas). El único requisito del cliente es que **ninguna letra del cartel puede quedar al lado de otra letra igual**, ya que esto genera un cortocircuito en el cableado de neón.

De todas las formas posibles de organizar las letras que cumplan esa condición, el cliente quiere la que se lea como la **palabra lexicográficamente mayor** (es decir, la que aparecería última en un diccionario).

Dada la cadena de letras disponibles, determine la organización lexicográficamente mayor tal que no haya dos letras iguales adyacentes.

Entrada

- Una única línea con una cadena S de letras latinas minúsculas ($1 \leq |S| \leq 10^6$).

Salida

- Una única línea con la reorganización lexicográficamente mayor de S tal que no haya dos caracteres iguales adyacentes.
- Si no es posible, imprima `Imposible`.

Restricciones

- Resolver usando una **estrategia greedy**.
- **Tiempo:** $O(N)$ donde $N = |S|$, considerando el alfabeto de 26 letras como constante.
- **Espacio:** $O(N)$.

Ejemplo

Input 1

```
aab
```

Output 1

```
aba
```

Explicación 1

Las letras disponibles son: `a`, `a`, `b`. Las posibles reorganizaciones son:

Reorganización	¿Válida?
aab	No (a y a adyacentes)
aba	Sí
baa	No (a y a adyacentes)

La única reorganización válida es **aba** . Resultado: **aba**.

Input 2

aaabbc

Output 2

cababa

Explicación 2

Letras disponibles: 3 **a** , 2 **b** , 1 **c** . La cadena **cababa** no tiene letras adyacentes iguales y es la mayor lexicográfica entre las reorganizaciones válidas. Alternativas lex mayores como **ccbbaa** o **cbabaa** quedan descartadas por tener repeticiones adyacentes.

Input 3

aaa

Output 3

Imposible

Explicación 3

Letras disponibles: 3 **a** . Para que tres **a** no queden adyacentes hacen falta al menos dos letras de relleno entre ellas (algo como **a?a?a**). Como no hay otras letras disponibles, no existe ninguna reorganización válida.

Input 4

abcabc

Output 4

```
cbcaba
```

Explicación 4

Letras disponibles: 2 `a`, 2 `b`, 2 `c`. La cadena `cbcaba` no tiene adyacencias iguales y es la mayor lexicográfica entre las válidas. Reorganizaciones lex mayores como `ccbbaa` o `cbcbbaa` quedan descartadas por sus repeticiones adyacentes.

Ejercicio 9 - Campamento dinámico

Descripción

Un amigo decidió organizar un campamento de varios días en una zona remota, lejos de la civilización, y te invitó a ser parte de la aventura. Como suele pasar, nadie quiere cargar peso de más... pero tampoco quieren olvidarse de nada importante.

Antes de partir, se reúnen para decidir qué objetos llevar. Tienen una lista de n objetos disponibles, y cada uno puede elegirse a lo sumo una vez. Cada objeto tiene un peso, un volumen, un costo, un valor de utilidad y una categoría (`supervivencia` o `comida`).

La mochila tiene restricciones logísticas estrictas: la selección final no puede exceder un peso máximo P , un volumen máximo V , ni un costo máximo C . Además, para probar sus habilidades en la naturaleza, se limita la cantidad de objetos por categoría: a lo sumo S de `supervivencia` y M de `comida`.

Entre todas las selecciones válidas (incluyendo la **mochila vacía**, que siempre cumple las restricciones con valor y peso cero), se debe elegir aquella que:

1. maximice el valor total de utilidad;
2. en caso de empate, minimice el peso total;
3. en caso de empate, minimice la cantidad de objetos seleccionados.

Entrada

- La primera línea contiene un entero n ($1 \leq n \leq 35$), la cantidad de objetos disponibles.
- La segunda línea contiene cinco enteros P , V , C , S y M ($1 \leq P, V, C \leq 20$, $0 \leq S, M \leq 15$): el peso máximo, el volumen máximo, el costo máximo, la cantidad máxima de objetos de supervivencia y la cantidad máxima de objetos de comida, respectivamente.
- Las siguientes n líneas describen un objeto cada una, con el formato:

```
peso volumen costo valor categoria
```

donde $1 \leq \text{peso}, \text{volumen}, \text{costo}, \text{valor} \leq 100$, y `categoria` es la cadena `supervivencia` o `comida`.

Salida

Imprima en una única línea tres enteros separados por espacio: el valor total máximo alcanzado, el peso total y la cantidad de objetos seleccionados de la solución óptima. Si la mejor solución es la mochila vacía, los tres valores son `0`.

Restricciones

- Resolver utilizando tabulación.
- La solución debe contemplar simultáneamente las restricciones de peso, volumen, costo y cantidades máximas por categoría.

Ejemplo

Input

```
5
10 12 11 2 2
3 2 4 8 supervivencia
2 2 3 6 supervivencia
2 3 2 5 comida
3 4 3 7 comida
4 3 5 9 supervivencia
```

Output

```
22 9 3
```

Explicación

Una selección válida es:

- objeto 2 (linterna): peso 2, volumen 2, costo 3, valor 6, supervivencia
- objeto 4 (arroz): peso 3, volumen 4, costo 3, valor 7, comida
- objeto 5 (manta): peso 4, volumen 3, costo 5, valor 9, supervivencia

Totales: peso 9, volumen 9, costo 11, valor 22. Se cumple peso ≤ 10 , volumen ≤ 12 , costo ≤ 11 , y hay a lo sumo 2 objetos de cada categoría. No existe otra selección con mayor valor total. La salida es `22 9 3`: valor 22, peso 9, 3 objetos.

Ejercicio 10 - Countdown

Descripción

Maurice Moss, fanático declarado del programa de televisión británico **Countdown**, ha decidido finalmente postularse como participante. Para prepararse, practica obsesivamente el famoso *numbers*

round: dado un conjunto de cartas con números y un número objetivo, debe combinarlos usando las operaciones aritméticas básicas (+ , - , * , /) para alcanzar exactamente dicho objetivo.

A diferencia del juego original, donde el participante puede elegir cuántas cartas usar, Moss quiere entrenarse con la regla más exigente posible: **todas las cartas deben ser utilizadas exactamente una vez**.

Cada operación toma dos números del conjunto y los reemplaza por el resultado de aplicarles la operación, dejando un número menos en el conjunto. El proceso se repite hasta que queda un único valor. Si ese valor coincide con el objetivo, la combinación es válida.

Dado un conjunto de cartas y un número objetivo, determine si es posible combinarlas para alcanzar exactamente el objetivo.

Entrada

- La primera línea contiene un entero n ($1 \leq n \leq 5$), la cantidad de cartas.
- Las siguientes n líneas contienen un entero cada una, el valor de cada carta ($1 \leq c_i \leq 100$).
- La última línea contiene el valor objetivo ($1 \leq T \leq 10^4$).

Salida

Imprima 1 si es posible alcanzar el objetivo utilizando todas las cartas exactamente una vez. Imprima 0 en caso contrario.

Restricciones

- Utilizar **backtracking** para explorar las combinaciones posibles.
- Todas las operaciones se realizan con **enteros**; cada resultado intermedio debe ser un **entero estrictamente positivo**:
 - La resta $a - b$ solo es válida si $a > b$.
 - La división a / b solo es válida si $b > 0$ y a es múltiplo de b (división exacta).

Ejemplo

Input 1

```
4
1
1
1
1
1
4
```

Output 1

```
1
```

Explicación 1

Las cartas son 1, 1, 1, 1 y el objetivo es 4. Una combinación válida que utiliza las cuatro cartas:

$$1+1+1+1=4$$

Resultado: 1.

Input 2

4
1
2
3
4
99

Output 2

0

Explicación 2

Las cartas son 1, 2, 3, 4 y el objetivo es 99. No existe ninguna combinación de las cuatro cartas con las operaciones +, -, *, / que dé como resultado 99. Resultado: 0.

Input 3

3
6
3
2
4

Output 3

1

Explicación 3

Las cartas son 6, 3, 2 y el objetivo es 4. Una combinación válida:

Paso	Operación	Cartas restantes
1	$6/3=2$	2, 2
2	$2*2=4$	4

Se utilizan las tres cartas y se alcanza el objetivo. Resultado: 1.

Input 4

5
3
8
2
1
5
24

Output 4

1

Explicación 4

Las cartas son 3, 8, 2, 1, 5 y el objetivo es 24 . Una combinación válida:

Paso	Operación	Cartas restantes
1	$8/2=4$	3, 4, 1, 5
2	$5-1=4$	3, 4, 4
3	$4+4=8$	3, 8
4	$3*8=24$	24

Se utilizan las cinco cartas y se alcanza el objetivo. Resultado: 1.

Formato de entrega (código fuente)

El formato de entrega del obligatorio va a constar de un archivo zip que en su interior contenga únicamente archivos con extensiones ".cpp" y ".h" en caso de C/C++ o ".java" en caso de utilizar Java. También podrán existir directorios para organizar mejor el código y poder reutilizar implementaciones en varios ejercicios.

Es importante que el archivo zip no contenga archivos de ningún otro tipo, como test/pruebas, archivos binarios, etc.

Recomendamos basarse en los templates provistos por la cátedra para minimizar la posibilidad de errores en la entrega.

- [Template C++](#)
- [Template Java](#)

C/C++

En el caso de C/C++, la estructura dentro del zip tendrá el siguiente formato:

```
|— directorioA
|— directorioB
...
|— directorioZ
|— ejercicio1.cpp
|— ejercicio2.cpp
...
|— ejercicio10.cpp
```

Por ejemplo, si tuviéramos 10 ejercicios, un posible zip a entregar podría tener:

```
|— funciones
|   |— enteros.cpp
|— tads
|   |— ListImp.cpp
|   |— List.h
|— ejercicio1.cpp
|— ejercicio2.cpp
...
|— ejercicio10.cpp
```

Notar que es obligatorio que en la raíz del zip se encuentren los archivos "ejercicioX.cpp". Los directorios para funciones auxiliares y TADs pueden llamarse de cualquier forma y no son obligatorios. Un ejemplo donde no se utilizan directorios auxiliares podría ser:

```
|— enteros.cpp
|— ListImp.cpp
|— List.h
|— ejercicio1.cpp
|— ejercicio2.cpp
...
|— ejercicio10.cpp
```

Cómo compilar

La forma de compilar cada ejercicio es la vista en el curso, pasando por parámetro el archivo raíz "ejercicioX.cpp", siendo X el número de ejercicio, e indicando en el parámetro "-o" el nombre del archivo compilado:

```
g++ -o ejercicioX ejercicioX.cpp -std=c++11
```

Cómo realizar las inclusiones

Las inclusiones parten del archivo "raíz" del programa, es decir, el "ejercicioX.cpp".

Dependiendo de si los archivos a incluir se encuentran en el mismo directorio o no, es cómo se realizará la inclusión.

Con directorios

Dado el siguiente caso:

```
|— funciones
|   |— enteros.cpp
|— tads
|   |— ListImp.cpp
|   |— List.h
...

```

Se definen las inclusiones de la siguiente manera:

```
#include "tads/List.h"
#include "tads/ListImp.cpp"
#include "funciones/enteros.cpp"
```

Sin directorios

Dado el caso en el que no se usan directorios:

```
|— enteros.cpp
|— ListImp.cpp
|— List.h
...
```

Se definen las inclusiones de la siguiente manera:

```
#include "List.h"
#include "ListImp.cpp"
#include "enteros.cpp"
```

Aclaración sobre TADs

Notar que en el archivo raíz se incluye tanto la especificación del TAD (List.h) como la implementación (ListImp.cpp). Lo correcto es incluir únicamente la especificación y asignar la implementación al momento de compilar. Esto resulta en problemas para realizar debugging y en que la compilación sea más compleja de estandarizar, por lo que se decide que es aceptable realizar la inclusión de la implementación.

JAVA

En el caso de Java, la estructura dentro del zip tendrá el siguiente formato:

```
|— directorioA
|— directorioB
...
|— directorioZ
|— Ejercicio1.java
|— Ejercicio2.java
...
|— Ejercicio10.java
```

Por ejemplo, si tuviéramos 10 ejercicios, un posible zip a entregar podría tener:

```
|— funciones
|  |— Enteros.java
|— tads
|  |— List.java
|— Ejercicio1.java
|— Ejercicio2.java
...
|— Ejercicio10.java
```

Notar que es obligatorio que en la raíz del zip se encuentren los archivos "EjercicioX.java". Los directorios para funciones auxiliares y TADs pueden llamarse de cualquier forma y no son obligatorios. Un ejemplo

donde no se utilizan directorios auxiliares podría ser:

```
|— Enteros.java
|— List.java
|— Ejercicio1.java
|— Ejercicio2.java
...
|— Ejercicio10.java
```

Cómo compilar

La forma de compilar cada ejercicio es la vista en el curso, pasando por parámetro el archivo raíz "EjercicioX.java", siendo X el número de ejercicio:

```
javac EjercicioX.java
```

Cómo realizar las importaciones

Las importaciones parten del archivo "raíz" del programa, es decir, el "EjercicioX.java".

Dependiendo de si los archivos a incluir se encuentran en el mismo directorio o no, es cómo se realizan.

Con directorios

Dado el siguiente caso:

```
|— funciones
|   |— Enteros.java
|— tads
|   |— List.java
..
```

Se definen las importaciones de la siguiente manera:

```
import tads.List;
import funciones.Enteros;
```

Para que esto funcione es importante definir los paquetes java de las clases que se encuentran en esos directorios.

Dado el ejemplo anterior, la clase Enteros.java debe definir su paquete al comienzo del archivo:

```
package funciones;
```

De la misma forma lo debe hacer la clase List.java:

```
package tads;
```

Las clases de la raíz, "EjercicioX.java" por ejemplo, no deben tener definido un paquete.

Sin directorios

Dado el caso en el que no se usan directorios:

```
|— Enteros.java  
|— List.java  
...
```

No es necesario definir ninguna importación. Es importante que ninguna clase del directorio raíz tenga definido un paquete al comienzo de la misma.

Documentación

El documento del obligatorio debe contener la siguiente contenido:

- Carátula con los datos de los estudiantes.
- Breve explicación de la solución de cada problema.
- Justificación de la complejidad temporal de cada problema (sólo en el caso que haya restricciones planteadas en el enunciado).
- Declaración de autoría para cada problema. Leer **este enlace**.
- Una tabla que resuma los resultados obtenidos en los casos de prueba provistos por la cátedra.
 - Los resultados posibles son:
 - Completo: Corren todas las pruebas.
 - Parcial: Corren parcialmente las pruebas.
 - En Proceso: Implementado pero no pasa ninguna prueba.
 - No Implementado: Ejercicio no implementado o no compila.
 - Ejemplo de tabla:

Problema	Resultado
1	Completo
2	Completo
3	En Proceso
4	No Implementado
...	...
10	Completo

Autoría

Los alumnos deben especificar las fuentes de información y recursos utilizados para la resolución de cada ejercicio. Esto incluye:

- Código discutido o desarrollado en clase.
- Recursos o ejemplos encontrados en la web. Se debe especificar la URL de la página web.
- Código generado por herramientas de inteligencia artificial generativa (IAG).

Importante: Es imprescindible citar correctamente todas las fuentes utilizadas. La omisión de esta información podría considerarse como plagio, lo cual conllevaría a acciones disciplinarias según las políticas de la universidad. La transparencia en la citación de fuentes es crucial para mantener la integridad académica del obligatorio.

Inteligencia Artificial Generativa

- Seguir las pautas de los docentes: Se deben seguir las instrucciones específicas de los docentes sobre cómo utilizar la IA en cada curso.
- Citar correctamente las fuentes y usos de IA: Siempre que se utilice una herramienta de IA para generar contenido, se debe citar adecuadamente la fuente y la forma en que se utilizó.
- Verificar el contenido generado por la IA: No todo el contenido generado por la IA es correcto o preciso. Es esencial que los estudiantes verifiquen la información antes de usarla.
- Ser responsables con el uso de la IA: Conocer los riesgos y desafíos, como la creación de “alucinaciones”, los peligros para la privacidad, las cuestiones de propiedad intelectual, los sesgos inherentes y la producción de contenido falso.
- En caso de existir dudas sobre la autoría, plagio o uso no atribuido de IAG, el docente tendrá la opción de convocar al equipo de obligatorio a una defensa específica e individual sobre el tema.

Links de interés

- [ORT: Uso de herramientas generativas de inteligencia artificial en la enseñanza](#)
- [ORT: La evaluación del aprendizaje en el marco del uso de la ia generativa](#)

Defensa

La defensa es obligatoria y eliminatoria. La no realización de la misma implica la pérdida de la totalidad de los puntos de los obligatorios.

La defensa es una instancia clave del proceso de evaluación, en la que el alumno debe demostrar una comprensión clara sobre los algoritmos, estructuras de datos y decisiones técnicas empleadas en la elaboración de los obligatorios. A continuación se detallan la modalidad y expectativas para esta evaluación.

Modalidad y Preparación

- **Forma Escrita:** La defensa comprende preguntas escritas sobre ambos obligatorios y se realizará el mismo día que el segundo parcial (ver calendario de eventos).
- **Duración:** La duración máxima de la defensa escrita será de 15 minutos.
- **Contenido:** Las preguntas se centrarán en la justificación de algoritmos, estructuras de datos, versiones y decisiones específicas tomadas en cada uno de los ejercicios realizados durante ambos obligatorios.

Dinámica de la Defensa

- **Entrega y Realización:** La defensa se llevará a cabo inmediatamente después de la entrega del segundo parcial.
- **No memorización:** El objetivo no es que el alumno memorice su código, sino que sea capaz de explicar y justificar las decisiones técnicas adoptadas, así como demostrar su comprensión del funcionamiento de los algoritmos y estructuras empleadas.
- **Segunda instancia oral:** En caso de considerarlo necesario, el docente podrá citar al alumno o grupo a una segunda instancia de defensa oral online, coordinada posteriormente.

Expectativas

Durante la defensa escrita, el alumno debe demostrar:

- Comprensión sólida sobre los algoritmos y estructuras utilizados.
- Capacidad de justificar decisiones técnicas y versiones elegidas.
- Claridad en la explicación sobre el funcionamiento general del código desarrollado.

Importante: Una demostración insuficiente de comprensión en cualquiera de estos puntos puede implicar una pérdida parcial o total de los puntos asignados a los obligatorios.

Consideraciones Importantes

- **Preparación Conceptual:** Se recomienda revisar previamente los algoritmos y estructuras empleadas en los obligatorios, así como las razones detrás de cada elección técnica tomada.
- **Defensa complementaria:** La defensa oral, en caso de realizarse, se coordinará de forma online en fecha y horario acordados entre docentes y alumnos involucrados.

RECORDATORIO: IMPORTANTE PARA LA ENTREGA

La entrega de los obligatorios será en formato digital online, a excepción de algunas materias que se entregarán en Bedelía y en ese caso recibirá información específica en el dictado de la misma.

Los principales aspectos a destacar sobre la **entrega online de obligatorios** son:

1. Ingresá al sistema de Gestión.
2. En el menú, seleccioná el ítem "Evaluaciones" y la instancia de evaluación correspondiente, que figura bajo el título "Inscripto".
3. Para iniciar la entrega hacé clic en el ícono correspondiente.
4. Ingresá el número de estudiante de cada uno de los integrantes y hacé clic en "Agregar". El sistema confirmará que los integrantes estén inscriptos al obligatorio y, de ser así, mostrará el nombre y la fotografía de cada uno de ellos. Una vez agregados todos los integrantes, hacé clic en "Crear equipo".

Cualquier integrante podrá:

- Modificar la integración del equipo.
 - Subir el archivo de la entrega.
5. Seleccioná el archivo que deseás entregar. Verificá el nombre del archivo que aparecerá en la pantalla y hacé clic en "Subir" para iniciar la entrega. Cada equipo (hasta 2 estudiantes) debe entregar **un único archivo en formato zip** (los documentos de texto deben ser pdf, y deben ir dentro del zip). El archivo a subir debe tener un **tamaño máximo de 40mb**.
 6. Cuando el archivo quede subido, se mostrará el nombre generado por el sistema, el tamaño y la fecha en que fue subido.
 7. El sistema enviará un e-mail a todos los integrantes del equipo informando los detalles del archivo entregado y confirmando que la entrega fue realizada correctamente.
 8. Podés cerrar la pestaña de entrega y continuar utilizando Gestión o salir del sistema.
 9. **La hora tope para subir el archivo será las 21:00** del día fijado para la entrega.
 10. La entrega se podrá realizar desde cualquier lugar (ej. hogar del estudiante, laboratorios de la Universidad, etc).
 11. Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con la Coordinadora de Cursos o Coordinación Adjunta antes de las 20:00h del día de la entrega, a través del mail: adjuntos_ei@ort.edu.uy.